5.5.4 Annotating your scenarios with tags

You've seen (in section 5.5.1) how you can organize your feature files into subdirectories that mirror your requirements' structure. But often it's nice to have other ways to categorize your scenarios. Fortunately, this is quite easy to do. Both JBehave and Gherkin let you add *tags* to scenarios.

For example, many projects store details about features or user stories in issue-tracking software such as Atlassian's JIRA. In this case, it's useful to relate a feature or an individual scenario back to the corresponding issue, both for information and so that reporting tools can use this data to create a link back to the corresponding issue. In JBehave, you can do this using the `Meta` keyword:

```
Earning extra points from Frequent Flyer status

Meta:
@issue FF-123

Scenario: Earning extra points on flights by Frequent Flyer status
...
```

Tags need to be introduced by the Meta keyword.

Tags start with @ and can be any text value.

> **TIP** When you're defining tags with the `Meta` keyword in JBehave, it's especially important not to leave out the title in the first line. If you forget to include this text, JBehave will treat the `Meta` statement as a title, which may result in errors in your living documentation.

In Gherkin, things are even simpler:

```
Feature: Earning extra points from Frequent Flyer status
  @issue FF-123
  Scenario Outline: Earning points on flights by Frequent Flyer status
  ...
```

Gherkin doesn't need the Meta keyword.

Tags are also a great way to categorize scenarios by other cross-functional concerns, to identify related parts of the system, and to help organize test execution. For example, you might want to flag all of the web tests, or mark certain tests as being slow, so that they can be grouped together during the automated build process:

```
Feature: Earning extra points from Frequent Flyer status
  @issue FF-123
  @web @slow
  Scenario Outline: Earning points on flights by Frequent Flyer status
  ...
```

Tags can be used to identify related cross-functional concerns.

This way, when executing the tests, you can configure a filter to only run the tests with a particular tag (or *without* a particular tag).[7]

Some BDD tools (Cucumber, in particular) also let you write *hooks*—methods that will be executed before or after a scenario with a specific tag is executed. This is a

[7] You'll see how to do this in chapter 6.